Aula 16 — Composição

> **Combinando comportamentos**

Como acrescentar um comportamento a diferentes implementações sem repetir suas regras nem modificar o cliente?

O pedido conhece um contrato

```
public interface Entrega {
    double calcularCusto(Pedido pedido);
}
```

Implementação	Custo neste recorte
`EntregaNormal`	`10.0`
`EntregaExpressa`	`25.0`

Cada objeto conhece sua regra. O custo não depende dos itens neste recorte.

UMA NOVA NECESSIDADE PARA AS DUAS MODALIDADES

O seguro é opcional e acrescenta R$ 5

Escolha	Cálculo esperado
Normal sem seguro	`10.0`
Expressa sem seguro	`25.0`
Normal com seguro	`10.0 + 5.0`
Expressa com seguro	`25.0 + 5.0`

O cálculo não cobra valores, não contrata serviços e não altera o pedido.

ATIVIDADE

ANTES DE ESCOLHER UMA IMPLEMENTAÇÃO

Onde cada regra deve ficar?

Proposta 1: copiar o custo de cada modalidade para novas classes `EntregaNormalComSeguro` e `EntregaExpressaComSeguro`.

Proposta 2: fazer `Pedido` identificar a modalidade e calcular o acréscimo.

1. Se o custo normal mudar, o que precisará ser atualizado na proposta 1?
2. Que conhecimento novo o pedido recebe na proposta 2?
3. Quais duas tarefas podem ser separadas?

SEPARAR A REGRA DA MODALIDADE DO ACRÉSCIMO

A entrega existente já sabe calcular a base

1. O custo normal estaria repetido em `EntregaNormal` e na classe que copiou sua fórmula. As duas precisariam acompanhar a mudança.
2. `Pedido` passaria a conhecer as modalidades e a regra do seguro.
3. Podemos separar **calcular o custo-base** de **acrescentar o seguro**.

O novo comportamento pode solicitar o cálculo a um objeto existente.

Uma entrega com seguro recebe sua base

```
public class EntregaComSeguro implements Entrega {
    private Entrega entregaBase;

    public EntregaComSeguro(Entrega entregaBase) {
        this.entregaBase = entregaBase;
    }
    // calcularCusto aparece no próximo slide
}
```

O construtor conserva a referência recebida. Considere colaboradores diferentes de `null`.

Solicitar o custo e acrescentar o seguro

Dentro de `EntregaComSeguro`:

```
public double calcularCusto(Pedido pedido) {
    double custoBase = entregaBase.calcularCusto(pedido);
    return custoBase + 5.0;
}
```

O objeto alcançado por `entregaBase` calcula a modalidade.

O objeto com seguro acrescenta `5.0` ao resultado recebido.

DUAS DECISÕES DIFERENTES NA MESMA CLASSE

Cumprir o contrato e guardar o colaborador

Declaração	Papel
`implements Entrega`	permite usar o objeto onde se espera uma entrega
`private Entrega entregaBase`	conserva outro objeto para solicitar o cálculo-base

O método de `EntregaComSeguro` executa uma chamada a **outro objeto**, pelo campo `entregaBase`.

ATIVIDADE

CONSULTAS DIRETAS ÀS ENTREGAS; O PEDIDO É SÓ ARGUMENTO

A mesma classe, duas combinações

```
Pedido pedido = new Pedido();
Entrega normal = new EntregaNormal();
Entrega normalSegurada = new EntregaComSeguro(normal);
Entrega expressaSegurada =
        new EntregaComSeguro(new EntregaExpressa());
System.out.println(normalSegurada.calcularCusto(pedido));
System.out.println(expressaSegurada.calcularCusto(pedido));
System.out.println(normal.calcularCusto(pedido));
```

1. Quais valores aparecem? Quem calcula a base na segunda impressão?
2. Criar `normalSegurada` alterou a entrega normal?

A COMBINAÇÃO USA A REGRA DO COLABORADOR ESCOLHIDO

15.0, 30.0 e 10.0

Consulta	Cálculo	Resultado
Normal segurada	normal devolve `10.0`; seguro soma `5.0`	`15.0`
Expressa segurada	expressa devolve `25.0`; seguro soma `5.0`	`30.0`
Normal diretamente	a própria normal devolve seu custo	`10.0`

Criar a combinação cria **outro objeto**, que guarda a normal. A regra e o estado da normal permanecem iguais.

CONCEITO-CHAVE

DAR NOME À COLABORAÇÃO OBSERVADA

Composição de comportamentos e delegação

Compor comportamentos: combinar o trabalho de objetos para realizar uma responsabilidade maior.

Delegar: solicitar a um colaborador parte do trabalho necessário.

Aqui, a entrega-base calcula sua modalidade; `EntregaComSeguro` acrescenta o seguro ao resultado.

RELACIONAR AO QUE ESTUDAMOS NA AULA 08

Guardar a base não estabelece posse exclusiva

Na Aula 08, o foco eram as partes estruturais de um todo, como os itens de um pedido.

Agora, o foco é a **combinação de comportamentos** por colaboração.

```
Entrega normal = new EntregaNormal();
Entrega segurada = new EntregaComSeguro(normal);
```

A variável `normal` ainda alcança a mesma entrega e permite usá-la diretamente.

ATÉ AQUI, CONSULTAMOS ENTREGAS DIRETAMENTE

Como fornecer a combinação ao pedido?

`EntregaComSeguro` cumpre `Entrega`. Pode ocupar o papel já esperado por `Pedido`:

```
Entrega base = new EntregaExpressa();
Entrega segurada = new EntregaComSeguro(base);
Pedido pedido = new Pedido();
pedido.fechar(segurada);
```

Vamos acompanhar o cálculo **depois desse primeiro fechamento válido**.

Pedido mantém sua implementação

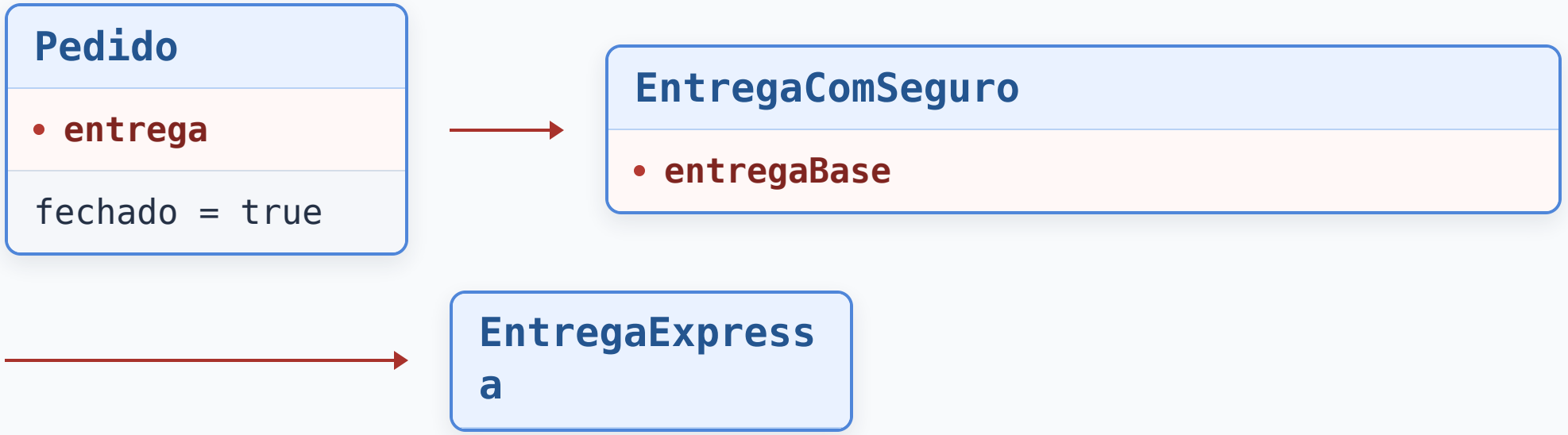
```
private Entrega entrega;
private boolean fechado;

public void fechar(Entrega entrega) {
    if (!fechado && entrega != null) {
        this.entrega = entrega;
        fechado = true;
    }
}
```

O pedido nasce aberto. O primeiro fechamento válido guarda a entrega recebida.

ESTADO APÓS PEDIDO.FECHAR(SEGURADA)

Os campos conservam o caminho da colaboração



`pedido`, `segurada` e `base` alcançam, respectivamente, esses três objetos.

As setas partem dos campos de referência. Não há cópia da entrega expressa.

ATIVIDADE

MESMO PEDIDO FECHADO COM A EXPRESSA SEGURADA

Reconstrua o cálculo completo

Dentro de `Pedido`:

```
public double calcularCustoEntrega() {
    return entrega.calcularCusto(this);
}
```

Agora, `Main` executa `pedido.calcularCustoEntrega()` e imprime o resultado.

1. Quais objetos recebem os dois cálculos pelo contrato `Entrega`?
2. Que valor cada entrega devolve e que valor será impresso?
3. A entrega-base recebe outro pedido ou o mesmo objeto?

MESMO CENÁRIO — SEGUIR AS CHAMADAS E AS DEVOLUÇÕES

O pedido devolve o resultado da combinação

```
Pedido solicita calcularCusto(this) à EntregaComSeguro
EntregaComSeguro solicita calcularCusto(pedido) à expressa
EntregaExpressa devolve 25.0
EntregaComSeguro acrescenta 5.0 e devolve 30.0
Pedido devolve 30.0; Main imprime 30.0
```

Os dois receptores são **EntregaComSeguro** e **EntregaExpressa**.

O argumento repassado é o **mesmo Pedido**: `this` foi recebido e encaminhado.

O polimorfismo participa das duas etapas

Onde está a chamada?	Referência usada	Objeto alcançado
`Pedido`	`entrega`	`EntregaComSeguro`
`EntregaComSeguro`	`entregaBase`	`EntregaExpressa`

O contrato permite cada chamada. O objeto alcançado executa a implementação da sua classe.

É assim que o mesmo pedido chega ao custo-base sem conhecer a modalidade.

ATIVIDADE

CONTINUE COM OS MESMOS OBJETOS E O PEDIDO FECHADO

Reatribuir base remonta a colaboração?

```
base = new EntregaNormal();
System.out.println(pedido.calcularCustoEntrega());

pedido.fechar(base);
System.out.println(pedido.calcularCustoEntrega());
```

1. Quais são os dois valores impressos?
2. O que aconteceu com `entregaBase` da combinação e `entrega` do pedido?

REATRIBUIÇÃO LOCAL E NOVO FECHAMENTO SÃO OPERAÇÕES DISTINTAS

Ambos continuam em 30.0

Operação	Efeito
`base = new EntregaNormal()`	muda somente a variável local
`pedido.fechar(base)`	a guarda recusa outro fechamento

`entregaBase` continua apontando para a **expressa**.

`entrega` continua apontando para a **combinação com seguro**.

A composição conserva as regras de estado que o pedido já possuía.

ATIVIDADE

DIAGNÓSTICO — SUBSTITUA SOMENTE O CORPO DE CALCULARCUSTO

Um resultado certo pode esconder uma falha

O construtor ainda recebe e guarda `entregaBase` nas duas propostas.

```
// A
return 10.0 + 5.0;
```

```
// B
Entrega normal = new EntregaNormal();
return normal.calcularCusto(pedido) + 5.0;
```

1. Que resultado cada proposta produz com normal? E com expressa?
2. Por que B falha, mesmo chamando outro objeto? Qual campo deveria usar?

As duas propostas sempre devolvem 15.0

Base recebida	Proposta A	Proposta B	Esperado
Normal	`15.0`	`15.0`	`15.0`
Expressa	`15.0`	`15.0`	`30.0`

B cria sua própria normal e ignora a escolha recebida.

A chamada necessária é `entregaBase.calcularCusto(pedido)`.

ATIVIDADE

RETOME A IMPLEMENTAÇÃO CORRETA; SURGE OUTRA MODALIDADE

E uma entrega agendada com seguro?

`EntregaAgendada` cumpre `Entrega` e devolve `15.0`, como na Aula 14.

1. Qual deve ser seu custo com seguro?
2. Que regra a modalidade oferece e como montar a combinação?
3. O que precisa mudar em `EntregaComSeguro` e `Pedido`?

A MODALIDADE OFERECE A BASE; A MONTAGEM FAZ A ESCOLHA

Agendada segurada custa 20.0

A modalidade implementa `calcularCusto(Pedido)` devolvendo `15.0`.

```
Entrega baseAgendada = new EntregaAgendada();
Entrega agendadaSegurada = new EntregaComSeguro(baseAgendada);
Pedido outroPedido = new Pedido();
outroPedido.fechar(agendadaSegurada);
```

`EntregaComSeguro` continua somando `5.0` à base; `Pedido` continua solicitando pelo contrato. Suas implementações permanecem iguais.

Um teste consulta o pedido fechado

Em `EntregaComSeguroTest`, com JUnit Jupiter configurado:

```
@Test
void acrescentaSeguroAoCustoExpresso() {
    Pedido pedido = new Pedido();
    pedido.fechar(new EntregaComSeguro(new EntregaExpressa()));

    assertEquals(30.0, pedido.calcularCustoEntrega());
}
```

A expectativa percorre pedido → combinação → entrega-base.

O acréscimo não deve se acumular

```
@Test
void consultasNaoAcumulamSeguroNemAlteramBase() {
    Pedido pedido = new Pedido();
    Entrega base = new EntregaNormal();
    Entrega segurada = new EntregaComSeguro(base);

    assertEquals(15.0, segurada.calcularCusto(pedido));
    assertEquals(15.0, segurada.calcularCusto(pedido));
    assertEquals(10.0, base.calcularCusto(pedido));
}
```

O seguro integra cada resultado; não é um saldo que cresce entre consultas.

ATIVIDADE

COMPARE OS DOIS TESTES COM AS PROPOSTAS A E B

Os dois cenários detectam os mesmos defeitos?

As versões defeituosas eram:

- **A:** devolver `10.0 + 5.0`;
- **B:** criar uma normal e solicitar seu custo, ignorando a base recebida.

1. O teste com **expressa segurada**, que espera `30.0`, detecta A e B?
2. O teste com **normal segurada**, que espera `15.0`, `15.0` e base `10.0`, também as detecta sozinho?

ESCOLHER OS DADOS DETERMINA O QUE O TESTE CONSEGUE DISTINGUIR

Com normal, as versões defeituosas parecem corretas

1. **Sim, o teste com expressa detecta as duas.** A e B devolvem `15.0`, mas a expectativa é `30.0`.
2. **Não, o teste com normal não as detecta sozinho.** As duas consultas devolvem `15.0`, e a normal original continua devolvendo `10.0`.

Um resultado esperado em um cenário não demonstra que a colaboração foi respeitada.

Fornecer uma entrega que registra a chamada

Em `EntregaRegistrada.java` — campos e construtor:

```
public class EntregaRegistrada implements Entrega {
    private double custo;
    private int quantidadeCalculos;
    private Pedido ultimoPedido;

    public EntregaRegistrada(double custo) {
        this.custo = custo;
    }
    // operação e consultas aparecem no próximo slide
}
```

Registrar antes de devolver o custo

```
public double calcularCusto(Pedido pedido) {
    quantidadeCalculos++;
    ultimoPedido = pedido;
    return custo;
}
public int getQuantidadeCalculos() {
    return quantidadeCalculos;
}
public Pedido getUltimoPedido() {
    return ultimoPedido;
}
```

O teste pode consultar a quantidade e a referência do pedido recebido.

Conferir resultado, quantidade e identidade

```
@Test
void delegaUmaVezComOMesmoPedido() {
    Pedido pedido = new Pedido();
    EntregaRegistrada base = new EntregaRegistrada(40.0);
    Entrega segurada = new EntregaComSeguro(base);
    double custo = segurada.calcularCusto(pedido);

    assertEquals(45.0, custo);
    assertEquals(1, base.getQuantidadeCalculos());
    assertEquals(true, base.getUltimoPedido() == pedido);
}
```

`==` verifica se as referências alcançam o mesmo pedido.

ATIVIDADE

MESMO TESTE: BASE DE 40.0, UM CÁLCULO E O MESMO PEDIDO

E se a combinação chamar a base duas vezes?

Uma versão solicita `calcularCusto(pedido)` **duas vezes** à base e devolve apenas o segundo resultado mais `5.0`.

A base de registro devolve `40.0` nas duas chamadas.

1. Qual seria o custo final? A expectativa `45.0` passaria?
2. Qual das outras expectativas falharia e com que valor observado?

RESULTADO IGUAL; COLABORAÇÃO DIFERENTE

O contador revela a chamada duplicada

1. O custo seria **45.0**. A primeira expectativa passaria.
2. `base.getQuantidadeCalculos()` devolveria **2**, em vez do `1` esperado. Essa expectativa falharia.

A identidade do pedido ainda poderia estar correta.

Observar a colaboração permite detectar um defeito que o custo final esconderia.

Retome o jogo da Aula 15

```
public interface Efeito {
    void aplicarEm(Personagem personagem);
}
```

O personagem começa com **50 pontos**. `Pocao` recupera **20**; `Armadilha` retira **10**.

Dentro de `Personagem`:

```
public void receber(Efeito efeito) {
    efeito.aplicarEm(this);
}
```

ATIVIDADE

TRANSFERÊNCIA — DELEGAR UM EFEITO E DEPOIS RECUPERAR 5

Como distribuir as duas ações?

Proponha `EfeitoComRecuperacao`, que recebe um `Efeito` na criação.

1. Que contrato deve cumprir para ser recebido pelo personagem?
2. Que colaborador guarda e o que solicita a ele?
3. Que ação solicita ao personagem **depois** da delegação?
4. Com personagens novos de `50`, qual vida resulta da combinação com poção? E com armadilha?

MESMO CONTRATO, OUTRA CONSEQUÊNCIA DA CHAMADA

Cumprir Efeito, delegar e recuperar

Decisão	Solução
Contrato público	`implements Efeito`
Colaborador guardado	campo `Efeito efeitoBase`, recebido no construtor
Primeira ação	`efeitoBase.aplicarEm(personagem)`
Ação seguinte	`personagem.recuperarVida(5)`

Poção: `50 → 70 → 75`. **Armadilha:** `50 → 40 → 45`.

O mesmo personagem recebido é repassado ao efeito-base.

Uma combinação que altera estado

```
public class EfeitoComRecuperacao implements Efeito {
    private Efeito efeitoBase;
    public EfeitoComRecuperacao(Efeito efeitoBase) {
        this.efeitoBase = efeitoBase;
    }
    public void aplicarEm(Personagem personagem) {
        efeitoBase.aplicarEm(personagem);
        personagem.recuperarVida(5);
    }
}
```

`void` permite delegação. O personagem controla a vida por suas operações públicas.

Usar a armadilha com recuperação

```
Personagem personagem = new Personagem();
Efeito armadilha = new Armadilha();
Efeito combinado = new EfeitoComRecuperacao(armadilha);

personagem.receber(combinado);
System.out.println(personagem.getVida()); // 45
```

O personagem recebe o objeto combinado pelo contrato; esse objeto delega a primeira ação à armadilha.

E se a ação for enviar uma mensagem?

O contrato do projeto continua sendo:

```
public interface Notificador {
    void enviar(String mensagem);
}
```

Um objeto pode acrescentar informação **antes** de solicitar o envio ao canal escolhido.

O lembrete continua responsável pela mensagem original e pela contagem dos avisos.

Construir um texto com prefixo

```
String prefixo = "[URGENTE] ";
String mensagem = "Revisar colaboração";
String mensagemDoEnvio = prefixo + mensagem;
```

- Resultado: `"[URGENTE] Revisar colaboração"`.
- O espaço final pertence ao prefixo.
- `mensagem` continua contendo `"Revisar colaboração"`.

Combinar sem repetir a regra-base

- Cumprir um contrato e guardar um colaborador desse contrato são papéis diferentes.
- A delegação usa o objeto recebido; a nova classe realiza seu acréscimo.
- O código de montagem escolhe os objetos; o cliente usa o contrato.
- Referências e regras de estado conservam a colaboração.
- Testes verificam resultados, argumentos e quantidade de chamadas.

Acrescentar informação antes do envio

Criar `NotificadorComPrefixo` no Projeto 2:

- Receber prefixo e destino; delegar **um único envio** por chamada.
- Usar a combinação em aviso alternativo, lista e canal inicial.
- Preservar `Lembrete`, os canais, a mensagem e os contadores.
- Observar texto e quantidade com `NotificadorRegistro` e ampliar os testes.

IA Nível 1 — Tutor. Entrega: somente os arquivos Java.